Rheinisch-Westfälische Technische Hochschule Aachen
Alessandro Berti

Phase Review: 3rd Sprint of the Implementation of "Conformance Predictions"

Group 4:
Richard Schuppener, Cem Colak

Contents

1 Summary of Sprint 3 **3**

2 Codebase Structure **3**

3 Testing **4**

4 User Guide **4**

5 Technical Documentation **4**
5.1 Codebase Architecture 5

6 Team settings and Workflow **5**
6.1 Team settings 5

7 Sprint review **6**
7.1 Richard Schuppener 6
7.2 Cem Colak 7

A Appendix **7**
A.1 CLI Walkthrough 7
A.2 Detailed requirements coverage 10
A.2.1 IO 10
A.2.2 Feature Extraction 11
A.2.3 Model Training 12
A.2.4 Evaluation 12
A.2.5 Config & Pipeline 13
A.2.6 Project Infrastructure & Testing 13
A.3 Visualizations 14

1 Summary of Sprint 3

In this sprint, we focused on implementing all the remaining functionalities required for a fully functional end-to-end pipeline for conformance predictions. We intended to use the final sprint to further refine and optimise these functionalities, so that we can deliver a well-aligned and polished software artifact that fulfils Alessandro's requirements in the most sophisticated way. We also tested the functionality of the whole artifact for all options and combinations of options, as we can finally say that the artifact is functioning as intended and appropriate for a third sprint.

- **Documentation & User Guide:** Complete function and module-level docstrings (Google format) were added across all modules (io, feature_extraction, model_training, evaluation, pipeline, visualization, config). The codebase documentation is now automatically generated via `pdoc` and a PDF version is available. Furthermore, a comprehensive User Guide and an updated general README were established.
- **ROC and PR Curves:** Added functions and plotting capabilities to compute ROC and Precision-Recall curves using scikit-learn metrics. These are now fully tested and integrated directly into the main CLI.
- **Feature Importance Evaluation:** Developed new evaluation functions to retrieve and visualize the most important features for the prediction models. This functionality and its corresponding visualization have been seamlessly integrated into the main CLI pipeline.
- **Resource Scaling Visualization:** Implemented functionality to visualize the relation between system resource usage (runtime and memory) and the size of the event log.
- **CLI Testing & Refactoring:** Implemented a test runner for the CLI and conducted extensive functionality testing. Addressed minor bugs, refactored classification options for process outcomes, and fixed issues regarding predictions on new data.

2 Codebase Structure

The codebase remained organized as in Sprint 2, so no further final changes.

3 Testing

Our testing modules are also complete and covering all the expectations stated in the project profile document. We have implemented unit tests for all the modules, and we have also implemented a test runner for the CLI called `cli_test_runner.py`, which allows us to test the whole pipeline with different options and combinations of options. We have also tested the functionality of the whole artifact for all options and combinations of options, as we can finally say that the artifact is functioning as intended and appropriate for a third sprint.

4 User Guide

The User Guide is complete and can be used as a concise reference for the CLI options and usage.

5 Technical Documentation

To ensure maintainability, we thoroughly documented our codebase using the following workflow:

- **Documentation Format:** Each module includes a high-level summary docstring, and every function contains Google-style docstrings describing its behavior, arguments (`Args:`), exceptions (`Raises:`), and return values (`Returns:`).
- **Generation & Access:** The HTML documentation is automatically built using `pdoc` via:

  ```
  pdoc --docformat google src/ -o docs/
  ```

  This can be opened interactively via `Code/docs/index.html` in a browser. Additionally, a compiled PDF version is available as `documentation.pdf` in the `Sprint 3` directory.

While comprehensively documenting all functions and matching the Google format was a highly demanding effort, it ensures the pipeline remains fully accessible and easy to extend.

5.1 Codebase Architecture

To illustrate the flow of data through our modular codebase, we designed a pipeline architecture diagram (shown in Figure 1). It maps out the stages from raw inputs (logs and models) through Data I/O, Feature Extraction, Model Training, and Evaluation/Visualization, highlighting the dependencies between the respective Python source modules.

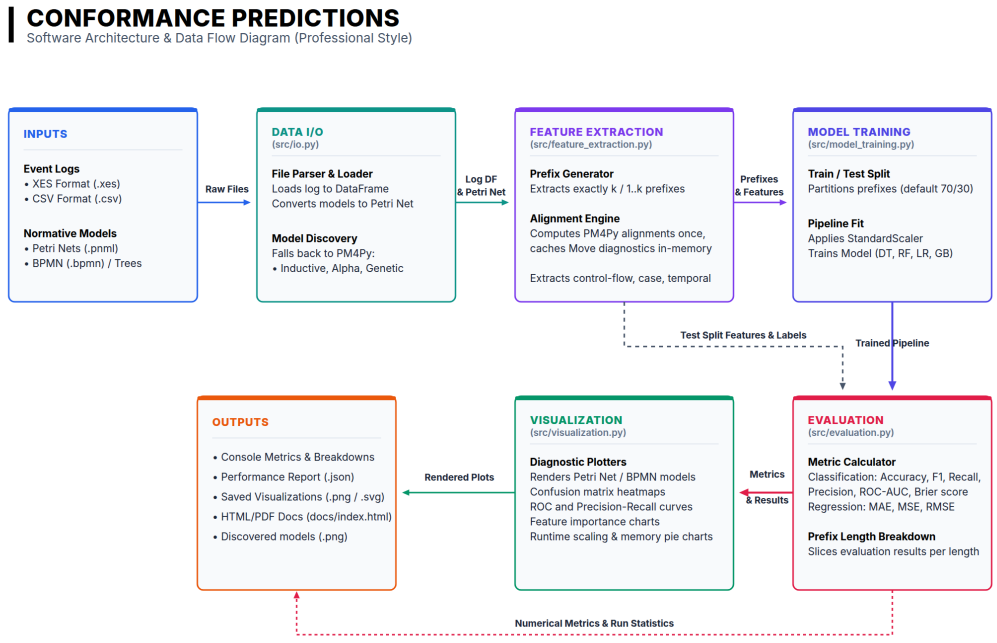


Figure 1: Data Flow and Pipeline Architecture Diagram

6 Team settings and Workflow

6.1 Team settings

Throughout the three sprints, we maintained a consistent rotation of responsibilities across all modules. Since the third sprint was more about adding the last functionalities we were focusing on documentation and finalization for the final sprint, which led to much more joint work. The documentation was a product

of boths contributions, and we also worked togehter on the CLI and config modules, as they are relatively small. The 2nd and 3rd sprint were also characterized by private and work-related boundaries regarding the workflow, as we both had travelled, other due submissions, and private commitments. We managed to work around these boundaries by supporting each other and by maintaing communication about our progress and challenges. The three sprints also helped us to improve our workflow and to find a sound balance between managing our time and orchestrating limitations. We also grew together, as we had to learn to adapt to each others working style and to find a common ground of understanding.

7 Sprint review

7.1 Richard Schuppener

I share Cem's sentiment that this final sprint was the most rewarding of the three. Since the core functionalities of the prediction pipeline were already in place, we could focus on refinement, polish, and thorough documentation. It was a well-timed and structured sprint, allowing both of us to balance this final push with our other upcoming university submissions due on June 14th.

My main focus during this sprint was on expanding our testing suite, refining the evaluation module, and implementing the remaining visualization capabilities (such as the resource scaling charts and prediction curves). I also performed some refactoring on the core components to ensure the codebase remains clean, modular, and easy to read. The most time-consuming and challenging task for me was definitely the technical documentation; writing and structuring standard Google-style docstrings for every single function and module across the codebase was highly demanding. Fortunately, I was able to dedicate a long train ride entirely to this task, which turned it into a productive and smooth writing session.

From a workflow perspective, our GitLab integration continued to be a major success. Using GitLab Work Items proved to be extremely helpful and descriptive for mapping out our tasks. Aligning this with our branching methodology—where we created a dedicated branch per work item that triggers the automatic merge pipeline and runs our comprehensive test suite—provided a robust and clean environment where we could merge with complete confidence.

The collaboration with Cem was, once again, flawless. His humor, commitment, and responsiveness made this lab project an exceptionally positive experience. We grew together as a team, and I am proud of the clean, fully documented, and well-tested software artifact we are delivering.

7.2 Cem Colak

This sprint was the most chill and straightforward one, as we had already implemented all necessary functionalities and were mostly focusing on finalizing the whole software artifact and on documentation. It was rather an intentional setting, because we both have submissions in other courses until the 14. June, and we wanted to have a more relaxed sprint to be able to focus more on the other submissions. Furthermore, I learned much more about the usage of Github, how to build a CLI, what to consider while building a software artifact, how to use ML algorithms in a prediction pipeline and how to document it. With Richard I have had an excellent partner, who dedicates to deliver and also has a great sense of humor, which made the whole project much more enjoyable. Additionally, he helped me a lot with understanding specific Process Mining concepts and with the implementation of the software artifact. I'm very happy with the outcome of our work and I think we have delivered a great software artifact considering the time limitations and the complexity of the task. Finally, I'm looking foward to evaluate the software artifact and finally finish the project with a good oral examination.

A Appendix

A.1 CLI Walkthrough

```
=== Conformance Predictions ===
Do you have a normative process model? (y/N):
Enter the path to the event log (XES/CSV): event_logs/sepsis.xes
Model discovery will be used.

Which discovery algorithm should be used?
1. Inductive Miner (recommended)
2. Alpha Miner
3. Genetic Miner
Select option (1-3): 1
Noise threshold (default 0.0): 0.01
Disable fall-throughs? (y/N):

Do you want a visualization of the discovered normative model saved
(s) or directly displayed (d) or not (N)? (s/d/N):
--- Loading event log: event_logs/sepsis.xes ---
/home/schuppener/spi/conformance-predictions/Code/.venv/lib/python3.14/site-packages/pm4py/u
-> UserWarning: Install the optional requirement ‘r4pm‘ to import/export files faster.
‘rustxes‘ remains supported as a fallback.
  warnings.warn(
parsing log, completed traces :: 100%|-------------| 1050/1050 [00:00<00:00, 3463.56it/s]

--- Discovering model using INDUCTIVE ---

--- Visualizing model ---

What should be predicted?
1. Future conformance deviations
2. Process outcome
3. Remaining time
Select option (1-3): 2

Define the target outcome:
Enter activity names that signify a outcome (one per line).
Type 'done' when finished.
> Release A
> Release B
> Release C
> Release D
> Release E
> done

What should the prediction be based upon?
1. Plain trace (control-flow)
2. Trace and alignment-based diagnostics
3. Case features
4. Mixture of all available information
Select option (1-4): 2

 Prefix mode (required for remaining time)?
   [Enter] varying 1..median (13)  (pooled - prefixes of length 1..k)
   1)      exactly k          (one prefix per case at length k)
```

```
 Select (Enter/1): 1
  prefix length k: 7

Enter a split value for the train- / test-split (default=0.7):
Invalid choice. Defaulting to 0.7
--- Generating prefixes (min=7, max=7, step=1, gate=off) ---
--- Extracting features over 628 prefixes: DIAGNOSTICS ---
aligning log, completed variants :: 100%|--------------| 143/143 [00:10<00:00, 13.15it/s]
--- Preprocessing and encoding data ---
--- Generating prefixes (min=7, max=7, step=1, gate=off) ---
--- Extracting features over 293 prefixes: DIAGNOSTICS ---
aligning log, completed variants :: 100%|--------------| 101/101 [00:06<00:00, 15.16it/s]
--- Preprocessing and encoding data ---

Which regression algorithm do you want to apply?
(1) Decision Tree
(2) Random Forest
(3) Gradient Boosting
Select option (1-3): 3
GRADIENT_BOOSTING selected.

Do you want to configure the parameters yourself? (y/N):

Training model...

Starting evaluation pipeline...
--- Using pre-trained model ---
--- Evaluating model ---
--- Feature Importance ---
                             importance  rank
sync_move_Admission NC         0.205193   1.0
sync_move_LacticAcid           0.193603   2.0
sync_move_Leucocytes           0.161959   3.0
sync_move_IV Liquid            0.145095   4.0
sync_move_IV Antibiotics       0.082441   5.0
sync_move_Admission IC         0.071276   6.0
sync_move_CRP                  0.063321   7.0
model_move_ER Sepsis Triage    0.041845   8.0
sync_move_ER Sepsis Triage     0.035268   9.0
sync_move_ER Registration      0.000000  10.5

Pipeline completed successfully!

Save feature importance chart? (y/N): n

Classification metrics (Global Pooled - Lengths: [np.int64(7)]):
  accuracy: 0.7142857142857143
  precision: 0.6649606417048278
  recall: 0.7142857142857143
  f1: 0.5990229771742377
  roc_auc: 0.4247287100556097
  brier_score: 0.0
```

```
Save Receiver Operating Characteristic and Precision-Recall curves? (y/N): y

The program consumed 206.6875 MB memory in peak.
---------------------------------------
The program took 0.6406 seconds to execute the following step:
Read and Discovery
---------------------------------------
The program took 0.0465 seconds to execute the following step:
Train/Test Split
---------------------------------------
The program took 9.2909 seconds to execute the following step:
Alignment & Feature Extraction (Train split)
---------------------------------------
The program took 7.5695 seconds to execute the following step:
Alignment & Feature Extraction (Test split)
---------------------------------------
The program took 0.4176 seconds to execute the following step:
Model Training
---------------------------------------
The program took 0.0220 seconds to execute the following step:
Evaluation
---------------------------------------
In total, it took 17.9871 seconds from start to finish (processing time only).

Do you want to visualize the processing times as a pie chart? (y/N): y

Do you want to save the runtime and memory measurements as JSON? (y/N): n

Do you want to perform predictions on new data? This will restart the CLI. (y/N): n
```

A.2 Detailed requirements coverage

A.2.1 IO

Covered

- **LL-01.1**: `io.discover_process_model()` supports Inductive, Alpha, and Genetic Miner via PM4Py, enabling automatic model discovery when no normative model is available.
- **LL-01.2**: `visualization.visualize_process_model()` renders the model as Petri net, BPMN, or Process Tree, with optional file saving or direct display.
- **HL-05**: Event log loading, model import, and model discovery are all functional; the foundational data-acquisition step of the pipeline is complete.
- **LL-05.1**: `pm4py.read_xes` for XES logs; `pd.read_csv` combined with `pm4py.format_dataframe` for CSV logs in `io.load_event_log()`.
- **LL-05.2**: Petri net loading via `pm4py.read_pnml` implemented; BPMN files loaded via `pm4py.read_bpmn` and Process Tree files via `pm4py.read_ptml`, both internally converted to a Petri net.

- **LL-05.3**: Inductive, Alpha, and Genetic Miner available as fallback discovery algorithms when no normative model file is provided.
- **HL-04**: `try/except` blocks in `main.py` catch and report setup errors, preprocessing failures, and pipeline exceptions to `stderr` with descriptive labels.
- **HL-01**: End-to-end prediction of conformance deviations is now fully operational; the *Future Deviations* target is wired through the complete pipeline – label extraction via `real_deviation_cost`, configurable threshold T, classification training, and evaluation.
- **LL-05.2**: Standalone loading of BPMN and Process Tree files as direct model inputs (without forced internal conversion to a Petri net).

A.2.2 Feature Extraction

Covered

- **HL-06**: `feature_extraction.compute_alignments()` invokes PM4Py's `conformance_diagnostics_alignments`; log-moves, model-moves, and synchronous moves are extracted per case. Distance-to-completion indicators are not yet included.
- **LL-06.1**: PM4Py alignment dictionaries are computed for every trace against the normative Petri net using the standard cost function. Alignments are now keyed by case ID and re-emitted in first-appearance order, eliminating the silent ordering issue that previously misaligned diagnostics with cases.
- **LL-06.2**: Alignment moves are parsed by `real_deviation_cost()` to count genuine deviations (log-moves and visible model-moves) per case; cumulative alignment cost is extracted while silent (τ) routing moves are correctly excluded.
- **HL-07**: `feature_extraction.extract_features()` now supports control-flow features (Bag-of-Activities), alignment-based diagnostic features (Bag-of-move-types per activity), case-attribute features, and the hybrid combination of all three.
- **LL-07.3**: Control-flow frequency matrix constructed via `pd.crosstab` over case IDs and activity names, producing a per-case Bag-of-Activities feature matrix.
- **HL-13**: Alignment results are computed once per run and stored in memory; they are passed as a variable to avoid redundant re-computation within a single pipeline execution.
- **HL-02**: Structured case attributes are fully integrated as predictive features. `list_case_attributes` and `summarize_case_attributes` detect both explicitly prefixed (`case:`) and constant-within-case attributes (XES-style), and the CLI lets the user select a subset of them.
- **LL-07.1**: `generate_prefixes()` expands each case into varying-length prefix pseudo-cases, with optional outcome-leakage gating, supporting both *exactly-k* and *varying* $1..k$ modes.
- **LL-07.2**: `extract_remaining_time()` and `extract_temporal_features()` together produce a numerical feature matrix tailored to the remaining-time regression target, including elapsed time, time since the previous event, and event count.
- **HL-06**: *We left this out.* Addition of distance-to-completion indicators (e.g. remaining enabled transitions, token distance to the final marking).
- **LL-13.1**: *We left this out.* Hash-based caching of prefix activity sequences to deterministically identify and skip redundant alignment computations.
- **LL-13.2**: *We left this out.* Disk-based persistence of alignment results via `joblib`.

A.2.3 Model Training

Covered

- **HL-08**: `train_classification_model()` wraps a configurable classifier (Decision Tree, Logistic Regression, or Gradient Boosting) in a `sklearn.Pipeline`. The binary deviation label is derived from the case's real alignment cost via the configurable threshold T, so the target is fully driven by alignment-based deviation cost.
- **LL-08.1**: `StandardScaler` is integrated as the first step in every sklearn Pipeline, ensuring scale-consistent feature distributions.
- **HL-09**: Baseline (control-flow features only) and conformance-aware (diagnostics or hybrid features) model variants are both supported via the `PredictionFeature` configuration.
- **LL-08.2**: Additional ML algorithms are available beyond the Decision Tree: *Logistic Regression* and *Gradient Boosting* for classification, and *Random Forest*, *Gradient Boosting*, and *Decision Tree Regressor* for regression. All are exposed via the CLI and selectable per run.

A.2.4 Evaluation

Covered

- **HL-16**: `evaluation.compute_classification_metrics()` computes accuracy, precision, recall, F1-score, ROC-AUC, and Brier score for binary and multi-class classification targets.
- **HL-12**: Confusion matrix heatmap and per-class accuracy bar chart are generated via Matplotlib in `visualization.save_evaluation_visuals()`.
- **HL-14**: The system allows for precise measuring of runtime and memory usage.
- **HL-14.1**: Each critical and potentially time-consuming step (often the pipelines) is precisely measured and the runtime is appended to running list. This functionality is performed by `measure_runtime`, which utilizes `time.perf_coutner` internally.
- **HL-14.2**: The peak memory consumed by the program is tracked by the function `measure_memory`. Additionally, it accounts for OS dependent units (bytes or kilobytes).
- **HL-14.3**: A counter accumulates the runtime from different steps in out program: from heavy *PM4Py* methods for discovery, test-train split etc., to whole pipelines that extract features or train models.
- **HL-14.4**: The generated performance data (runtime, peak memory usage) is printed to the CLI and can optionally be saved to a JSON. The runtime can also be visualized as a pie-chart.
- **HL-15**: The main pipeline uses `pm4py.split_train_test` with a user-configurable train ratio (default 0.7).
- **LL-16.2**: `compute_mae`, `compute_mse`, and `compute_rmse` are available in the evaluation module for evaluating remaining-time regression targets.
- **HL-17**: `evaluation.metrics_by_prefix_length()` slices the predictions of a single pooled model by prefix length and reports the full metric set per length; the CLI prints this table whenever prefix lengths vary.
- **HL-16**: *We left this out.* Conformance quality metrics for the normative model (fitness), and wiring of the regression metrics into the pipeline's `compute_regression_metrics` hook so they are reported automatically for the remaining-time target.

A.2.5 Config & Pipeline

Covered

- **HL-03**: `main.py` provides an interactive CLI that guides the user through model path/discovery selection, prediction target (future deviations, process outcome, remaining time), feature basis (including case attributes), prefix mode, train/test split ratio, and ML algorithm configuration.
- **LL-03.1**: Prediction target and feature basis options are fully defined in `config.py` and selectable in the CLI; both the case-attribute feature path and the conformance-deviation target are now fully wired through the pipeline.
- **HL-10**: `main.py` combined with `pipeline.py` forms an end-to-end runner from raw data and model input to evaluation output, without manual intermediate steps.
- **HL-14**: Runtime and resource measurement now wrap conformance checking, feature extraction, model training, and evaluation; the totals are reported at the end of every run.
- **LL-14.1**: `evaluation.measure_runtime()` uses `time.perf_counter()` around alignment computation, feature extraction, model training, and evaluation.
- **LL-14.2**: `evaluation.measure_memory()` polls `psutil.Process().memory_info()` to track peak RSS during the entire pipeline run.
- **LL-14.3**: Sub-second precision wall-clock measurements are recorded per step (conformance checking, feature generation, training, evaluation).
- **LL-14.4**: Runtime and resource metrics are printed to the console and can be persisted to a structured JSON log file at the end of the run.

A.2.6 Project Infrastructure & Testing

Covered

- **HL-18**: PM4Py, scikit-learn, pandas, NumPy, and Matplotlib are used as the primary library stack throughout all modules.
- **HL-19 & LL-19.1-19.2**: The project is organized into `io`, `feature_extraction`, `model_training`, `evaluation`, `pipeline`, `visualization`, and a centralized `config` module; Git is used for version control. Note: the conformance logic is consolidated into `feature_extraction` rather than placed in a dedicated `conformance` module. A formal branching strategy per module was not adopted, as we are working with one branch per team member.
- **HL-20**: `requirements.txt` and `README.md` are present in the repository.
- **LL-20.1**: `requirements.txt` as a plain text file is implemented and considered satisfactory for dependency management.
- **HL-11**: An automated pytest test suite covering IO, feature extraction, model training, and evaluation is executable via a single command (`pytest tests/`) and runs in the GitLab CI pipeline on every push or merge.
- **LL-11.3**: `test_generate_prefixes_lengths_and_content` asserts that a prefix of length k contains exactly the first k events of the original trace; the related `test_pooled_extraction_broadcasts_labels_without_leakage` test additionally confirms that outcome activities are gated out of the feature vector.

- **LL-11.1**: Dedicated pytest fixtures providing a synthetic, fully conforming XES log and a matching Petri net (currently the tests use ad-hoc DataFrame helpers).
- **LL-11.2**: Explicit test assertion that the alignment cost equals 0.0 on a fully conforming synthetic dataset.
- **HL-21**: Technical documentation of system architecture.

A.3 Visualizations

As mentioned in the previous sprint review document, we devoted a great part of our time to create meaningful and insightful visualizations to help us with the evaluation and the user with understanding the effect of the pipeline. As of Sprint 3, we implemented three more visualizations: a resource scaling plot, "Precision-Recall" and "Receiver Operating Characteristic" curves and a feature importance comparison plot.

Covered

- **Process Model Visualization**: `visualize_process_model` supports interactive viewing or saving of normative models (either discovered or provided) in three formats: Petri Net, BPMN, and Process Tree.
- **Decision Tree Visualization**: `visualize_decision_tree` renders the fitted scikit-learn Decision Tree (classifier or regressor), clearly showing the splitting criteria, node purity, and leaf values. This is crucial for the "white-box" interpretability requirement. An example output can be found in 3.
- **Confusion Matrix Heatmap**: `plot_confusion_matrix_image` generates a clear, color-coded heatmap (blues colormap) of the confusion matrix for classification tasks. The matrix contrasts predicted vs. actual outcomes, highlighting true positives and false positives. An example output can be found in 2.
- **Per-Class Accuracy Chart**: `plot_per_class_bar_chart` complements the confusion matrix by rendering a stacked bar chart displaying the percentage of correct (green) versus incorrect (red) predictions for each class label. An example output can be found in 4.
- **Processing Times Distribution**: `plot_processing_times` creates a pie chart that visualizes the proportion of time spent in each pipeline phase (e.g., IO, Alignment, Model Training). An example output can be found in 5.
- **Performance by Prefix Length**: `plot_evaluate_by_prefix_length` plots prediction metrics (Accuracy, F1 Score, or MAE for regression) on the Y-axis against the prefix length on the X-axis. This visually demonstrates whether adding more events to a trace improves the model's predictive power. An example output can be found in 6 and 7.

Implemented in Sprint 3

- **Feature Importance Comparison**: `plot_feature_importance` 10 renders a bar chart of the top predictive features, color-coding conformance-derived features to visually separate their impact versus standard control-flow features.
- **ROC and PR Curves**: `plot_roc_curves` 9 and `plot_pr_curves` 8 implement the overlay of curves of baseline vs. conformance-aware models, providing a standard ML evaluation visualization for binary risk targets.

- **Resource Scaling Plot**: `plot_runtime_log_size_relation` plots runtime and memory usage as a function of log size (number of cases/events) to empirically demonstrate the computational overhead of alignment extraction and other computationally expensive pipeline steps. This visualization will be included in the evaluation.

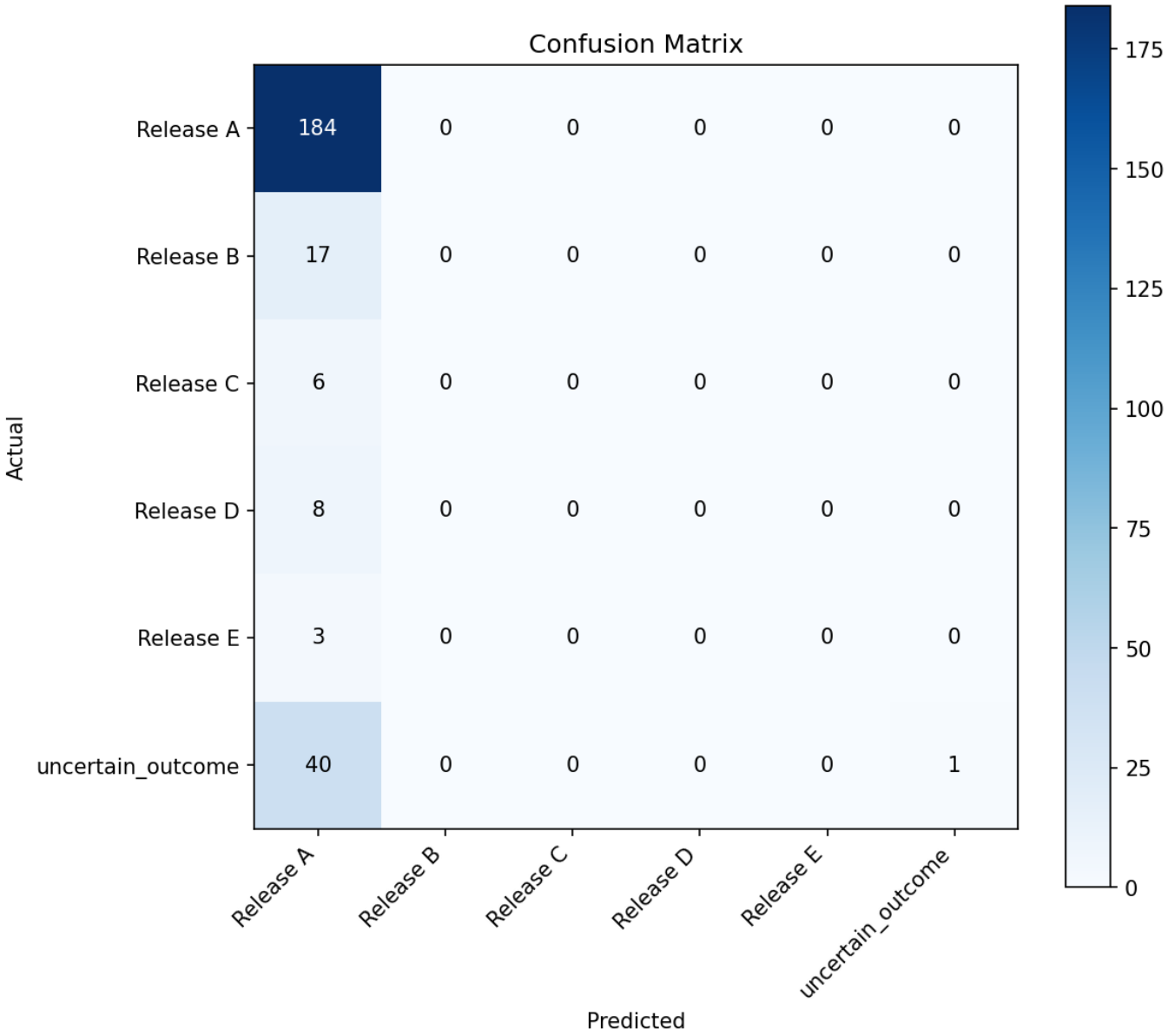


Figure 2: Confusion matrix visualization.

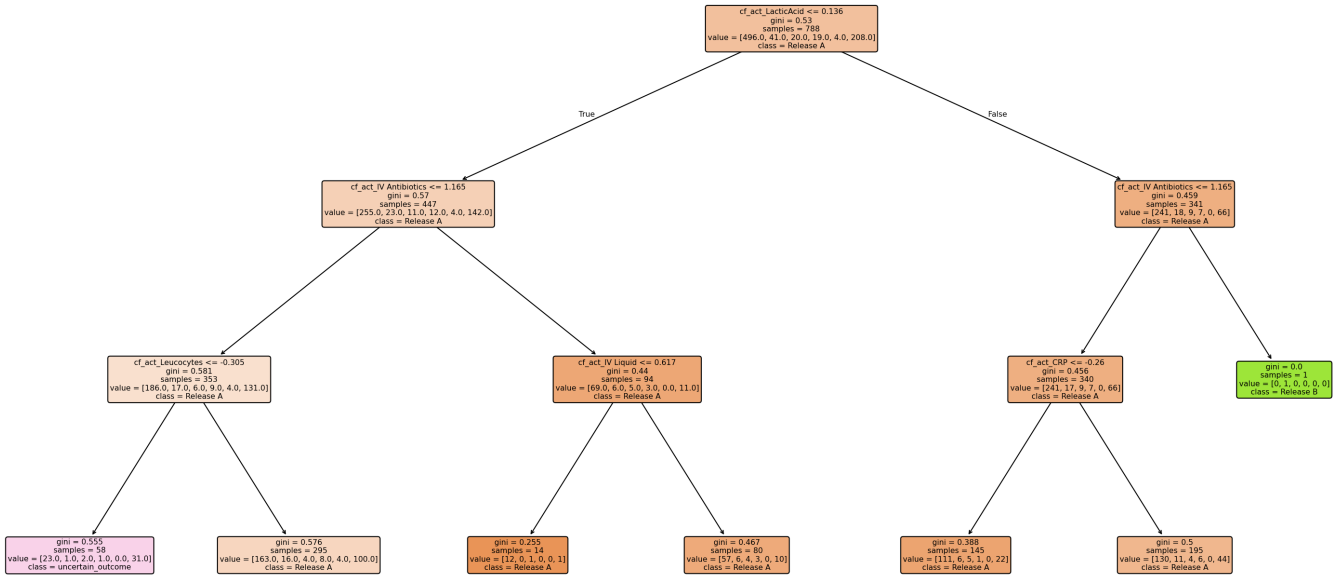


Figure 3: Decision tree visualization.

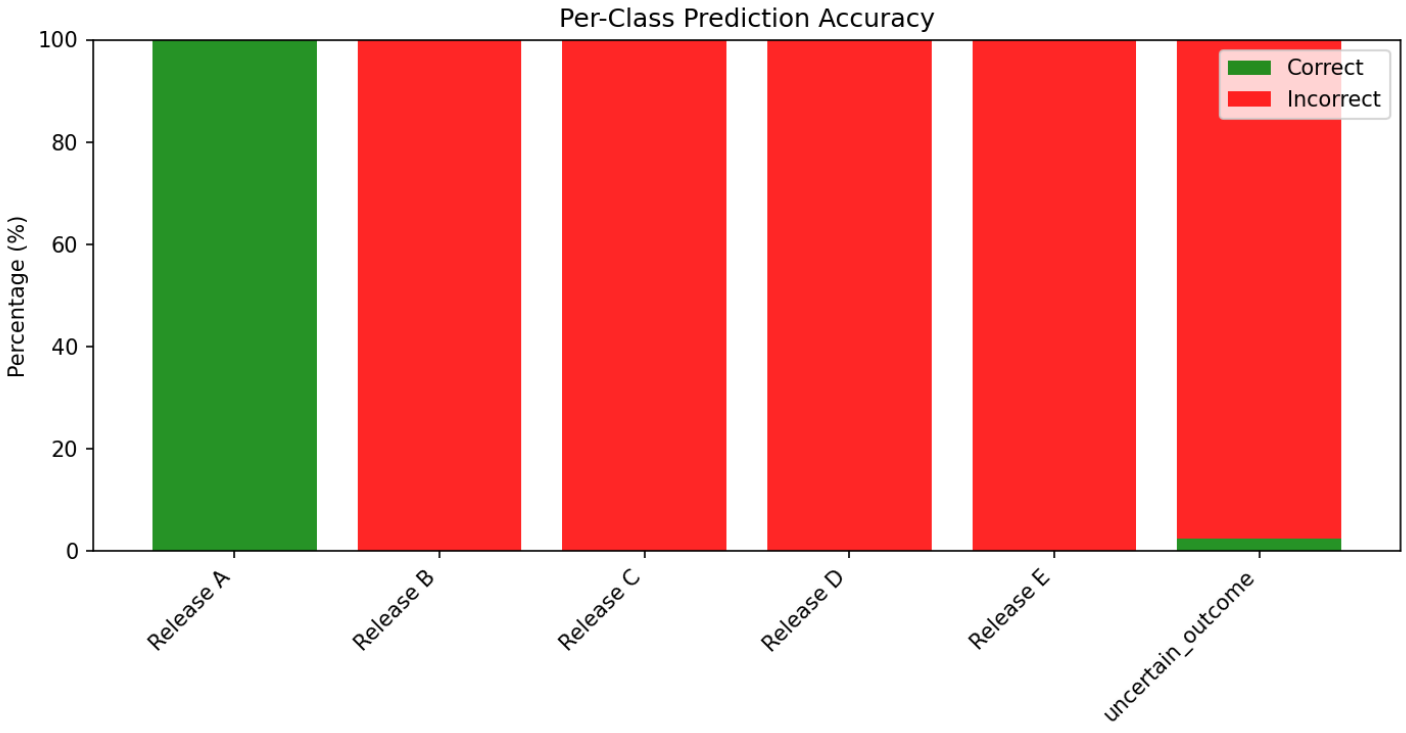


Figure 4: Accuracy per class graph.

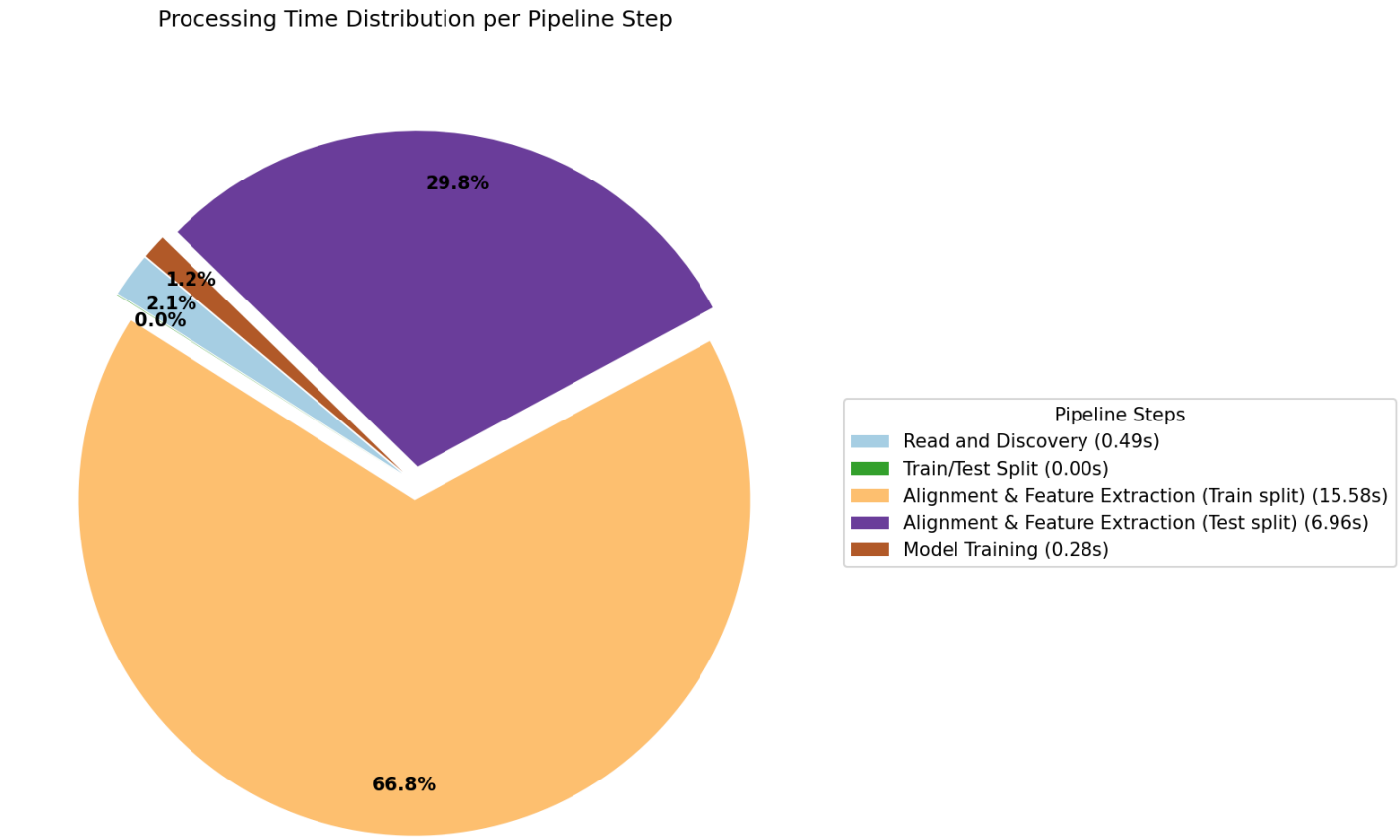


Figure 5: Processing Times Distribution as Pie-chart.



Figure 6: Performance by Prefix Length: Classification



Figure 7: Performance by Prefix Length: Regression

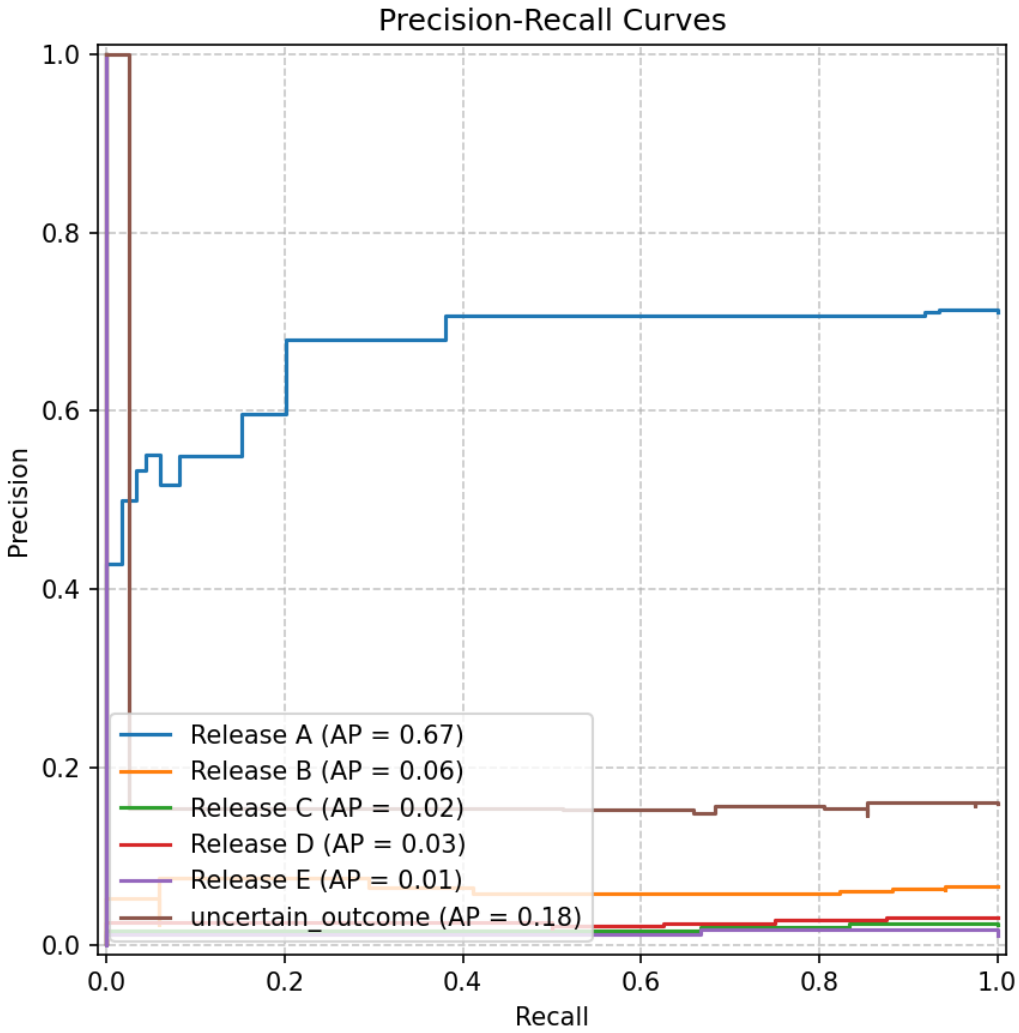


Figure 8: Precision-Recall curves.

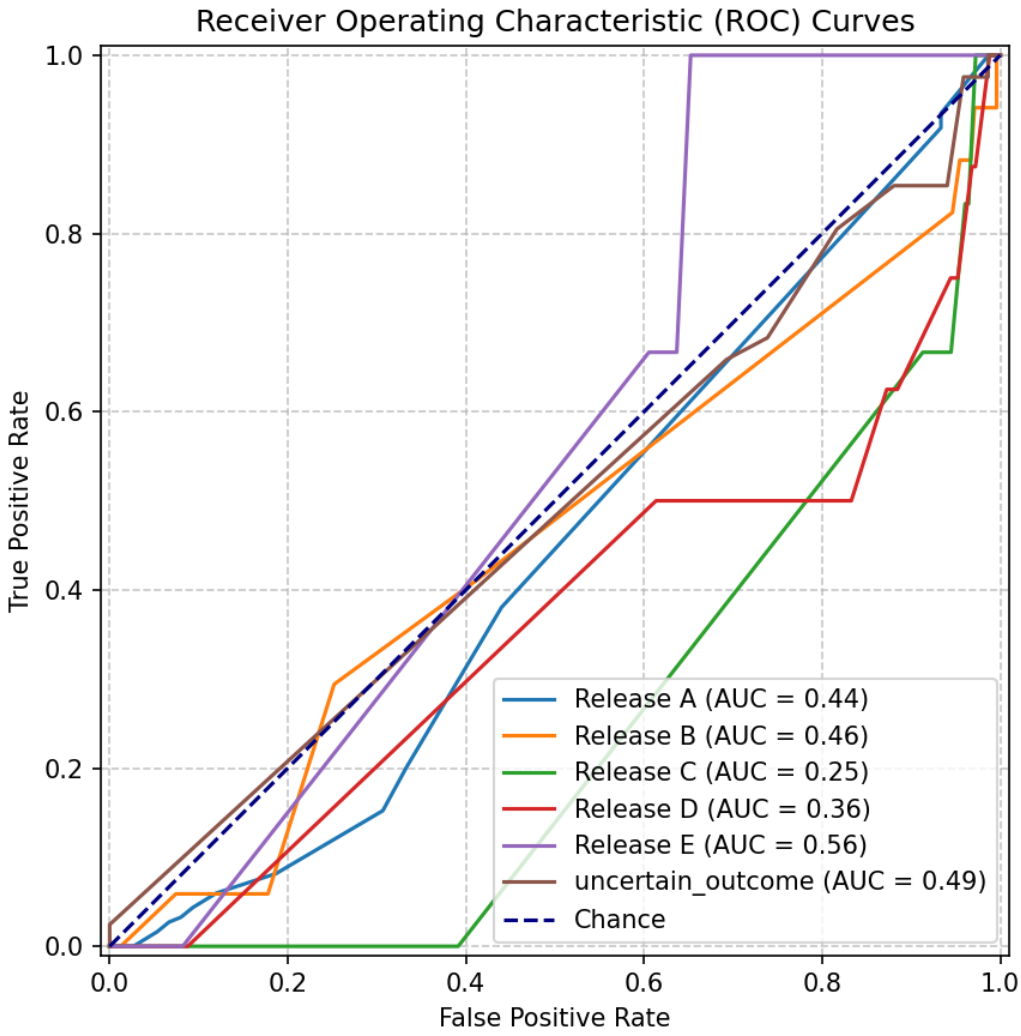


Figure 9: Receiver operating characteristic curves.

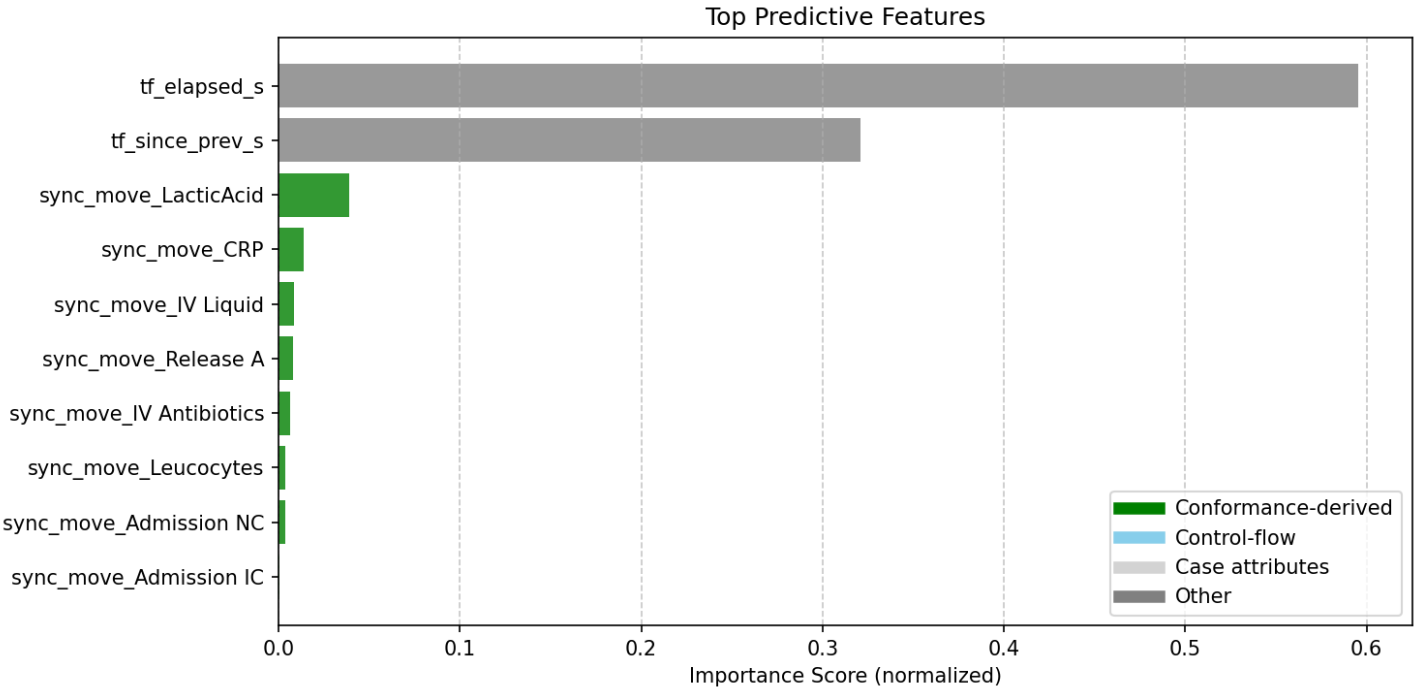


Figure 10: Bar-chart plotting the importance of utilized features for the prediction.